

Module Title

Cybersecurity and Networking

Internship Assignment Submission

SQL INJECTION FUNDAMENTALS & SAFE LAB REPORT

Student Name: Jiwosh Shrestha

Internship ID: JVL-CS-2026-8990

Role: Cybersecurity Intern

Duration: 3 months

Assignment Submission Date: Sunday, 18th October 2026

Day:28 (Twenty-Eight)

Table of Contents

1.DAY 28 ASSIGNMENT SQL INJECTION FUNDAMENTALS & SAFE LAB REPORT .. 3

1.1 PART A — FUNDAMENTALS 3

1.2 PART B — PORTSWIGGER LAB: RETRIEVAL OF HIDDEN DATA..... 5

1.3 PART C — DEFENSIVE ANALYSIS Fictional insecure design:..... 7

1.4 PART D — SHORT REFLECTION 8

1.DAY 28 ASSIGNMENT SQL INJECTION FUNDAMENTALS & SAFE LAB REPORT

1.1 PART A — FUNDAMENTALS

1.What is SQL?

SQL stands for structured Query Language. It is used to communicate with databases and store, retrieve, update or delete data.

2. What is SQL Injection?

SQL Injection is a security vulnerability where unsafe input changes the meaning of a database query. It may allow attackers to access data without permission.

3. What is a WHERE clause?

A WHERE clause is used to filter database records based on a specific condition. For example, it can display only products belonging to the Gifts category.

4. Explain AND and OR in SQL conditions.

AND means both conditions must be true, while OR means at least one condition must be true for the combined to be true.

5. Why is 1=1 considered an always-true condition?

Because the 1 is equal to itself, the condition 1=1 is always true.

6. What is SQL comment syntax used for conceptually?

SQL comments tell the database to ignore certain parts of query. In SQL Injection, comments can sometimes prevent the remaining query text from causing errors.

7. Explain why raw user input should not be concatenated directly into SQL queries.

Raw user input may change the query's meaning and allow unauthorized access to data. Parameterized queries help keep user input separate from SQL instructions.

8. What is a parameterized query?

A parameterized query separates SQL instructions from user-provided values. This helps prevent user input from being treated as SQL code.

9. What is a prepared statement?

A prepared statement is a database query prepared separately from the values supplies by the user. It helps execute queries safely and reduce SQL Injection risks.

10. Why does Least Privilege matter for database accounts?

Least Privilege means giving a database account only the permissions it needs. This limits the possible damage if the account or application is compromised.

1.2 PART B — PORTSWIGGER LAB: RETRIEVAL OF HIDDEN DATA

1. Which parameter was vulnerable? The HTTP GET category parameter in the product filter request (e.g., /filter?category=Gifts).

2. What product category did you initially select? Gifts

3. What was the purpose of the released = 1 condition? It restricted the product listing to only display items that were publicly released and marked as available (released = 1), hiding unreleased items (released = 0).

4. What controlled training input changed the query behavior? Gifts'+OR+1=1--

5. What does the 1=1 condition mean? It creates a tautology (an always-true boolean statement) that forces the entire WHERE clause to evaluate to true for every product in the database.

6. Why did additional/unreleased products become visible? Because the injected OR 1=1 clause forced the query to match all rows regardless of release status, while the trailing -- commented out the original AND released = 1 restriction.

7. Which Burp Suite feature did you use to resend the request? Burp Repeater (used to intercept, modify, and iteratively send the GET request with the injected payload).

8. What changed between the original response and modified training response? The original response returned only 3 released items under 'Gifts'. The modified response returned all products stored in the database, including unreleased products across all categories.

9. Did PortSwigger mark the lab as Solved? Yes, PortSwigger displayed the confirmation banner: "Congratulations, you solved the lab!".

10. Screenshot showing successful lab completion:


Lab: SQL injection vulnerability
SQL injection vulnerability in W
https://0af10046047e1bfb80d2cb87002c00db.web-security-academy.net/filter?category=Gifts
Web Security Academy
SQL injection vulnerability in WHERE clause allowing retrieval of hidden data
Back to lab description
LAB Solved
Congratulations, you solved the lab!
Share your skills!
Continue learning
Home

1.3 PART C — DEFENSIVE ANALYSIS Fictional insecure design:

User Input ↓ Raw String Concatenation ↓ SQL Query ↓ Database

Answer:

1.What is wrong with this design?

The design directly combines raw user input with SQL query text. This can allow malicious input to change the query logic and expose or modify data without permission.

2. Rewrite the workflow conceptually using a parameterized query.

User Input – parameterized query – Database Execution – Results

The query structure remains separate from user input, so the input is treated as data rather than SQL instructions.

3. Why should input also be validated server-side?

Server-side validation checks whether the input has the correct type, length, range and format. It helps reject invalid or unexpected values before processing them.

4. Why should the database account use Least Privilege?

The database account should have only the permissions required for its tasks. This reduces the damage that could occur if an attacker exploits the application.

5. Why should detailed database errors not be exposed publicly?

Detailed errors may reveal sensitive information such as database queries, table names, and internal server details. Users should receive simple error messages, while technical details should be stores securely in logs.

1.4 PART D — SHORT REFLECTION

Write 5–8 sentences explaining: What did this lab teach you about the difference between user DATA and SQL QUERY STRUCTURE?

This lab taught me that user input should be treated as data, not as SQL instructions. SQL Injection happens when unsafe input changes the structure or behavior of a database query. In the PortSwigger lab, I learned how a vulnerable category parameter could change the products displayed by the application. I also learned how Burp Suite Repeater can be used to resend and compare requests in an authorized testing environment. Server-side validation and Least Privilege provide additional protection. Overall, I understood why secure query handling is important for protecting database information.